Computer Graphics Using OpenGL
INM376 / IN3005

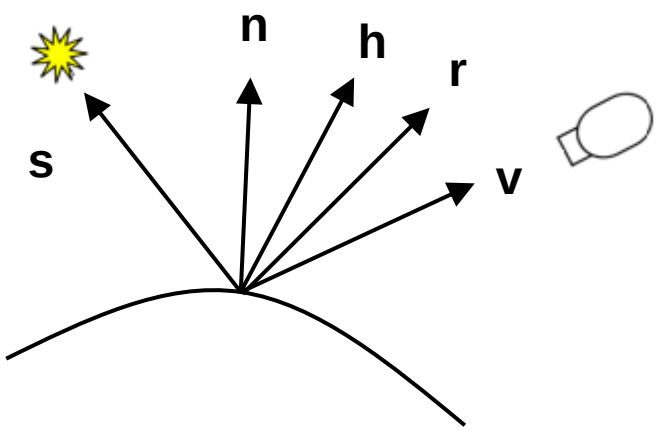
Module Leader – Dr Eddie Edwards

Lecture 07: Screenspace techniques

Recap from last time

- Heightmap terrain
- Lighting
  - Light sources
  - Materials
  - Reflectance models

$$\mathbf{c} = \mathbf{l}_a . * \mathbf{m}_a + \mathbf{l}_d . * \mathbf{m}_d \left[\max\left(\mathbf{s} \cdot \mathbf{n}, 0\right)\right] + \mathbf{l}_s . * \mathbf{m}_s \left[\max\left(\mathbf{h} \cdot \mathbf{n}, 0\right)\right]^n$$



Questions

- Can I leave out part of the coursework if I do more in other sections?
  - No – if you leave a part out you will not get the marks for that part. It's that simple
  - No-one gets 100% for any significant parts of the CW

- Does the player have to follow the path?
  - No – you can have free movement, perhaps restricted to the ground plane or track. BUT…
    - The interaction is more complicated than left/right motion
    - Restricting the player to the track edges is more complicated
    - Gameplay may be more difficult (for the developer and player)

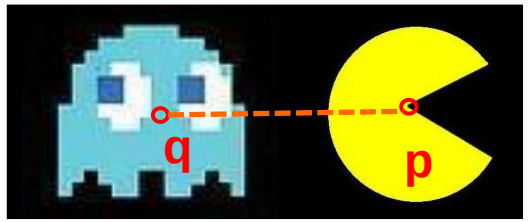
Questions

- Question: How do I know if my player is on the path?
- Answer: When you made your path, you defined its size; for example, using a width *w*. If you're moving your player left and right, you can keep track how far you've moved the player from the centreline.
  - If the player's horizontal distance is > *w/2.0f*, the player is off the right side of the path
  - If the player's horizontal distance is < -*w/2.0f*, the player is off the left side of the path

Questions

- Question: How do I do collision detection between the player and an obstacle / pickup?
- Answer: There are many techniques for collision detection. In this module, my advice is to *keep it simple*. The simplest form of collision detection: `glm::length()`

- If the distance between two objects is below a threshold T, register a collision.




```
if (glm::length(p-q) < T)
    // collision
```

- This requires that you know **p** and **q**. If you have a large number of obstacles / pickups, store them in an `std::vector`. Then, when performing collision detection apply the above algorithm for each object **q** with the player **p**.

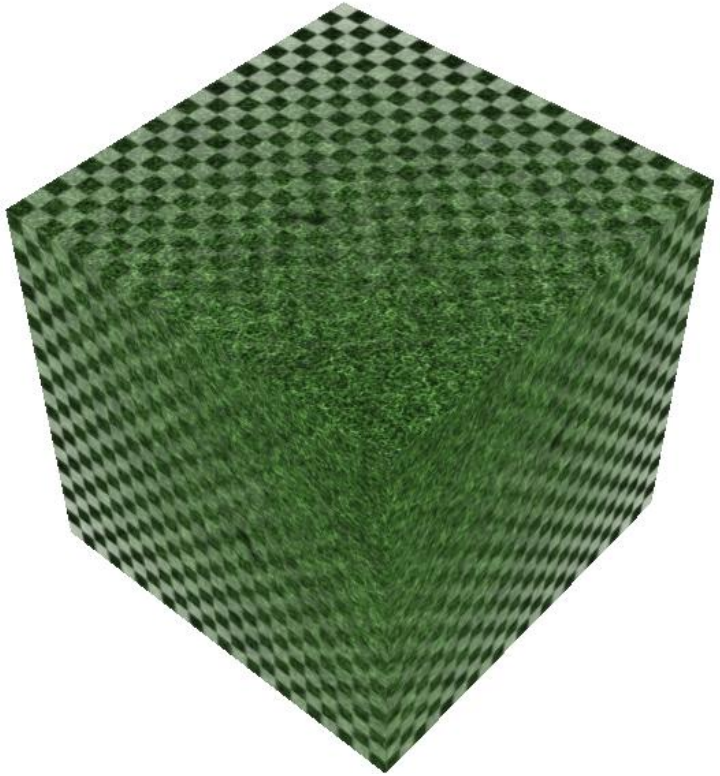
⇨ A natural place in the template code to do collision detection is `Game::Update()`

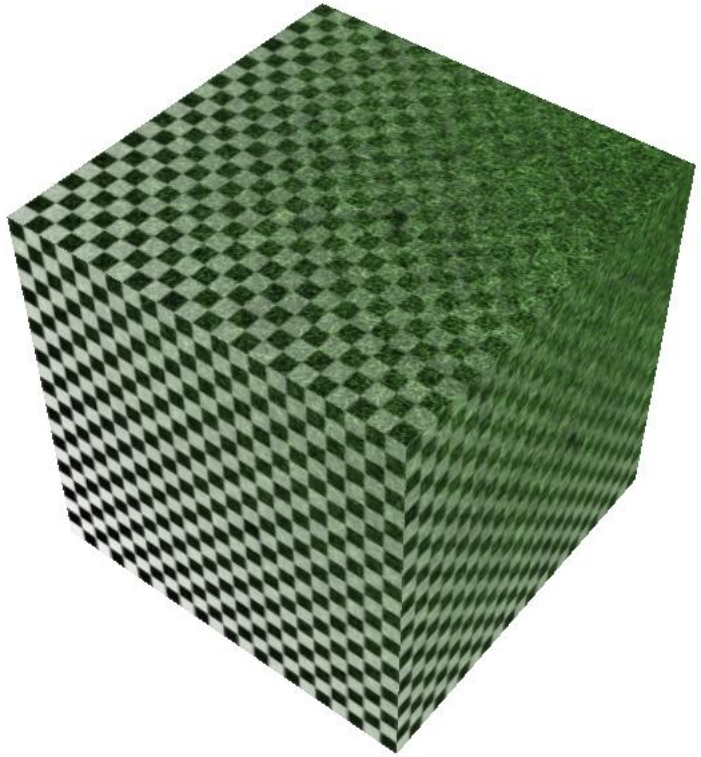
Overview of today's lecture

- Multi-texturing
- Buffers
  - Colour
  - Depth
  - Stencil
  - Framebuffer objects (FBOs) for offscreen rendering
- Image processing
  - Render to texture
  - Edge detection
  - Blur
  - Bloom
  - Post-processing effects
- Noise

Multi-texturing

- Modern GPUs support the capability to apply two or more textures simultaneously during a single pass rendering.
- Example: Grass texture and checkerboard texture





Multi-texturing

- First, it is a good idea to determine how many texture units are supported by the graphics hardware. This can be done using

```
GLint MaxUnits;
glGetIntegerv(GL_MAX_TEXTURE_IMAGE_UNITS, &MaxUnits);
```

- On my computer, there are 32 texture units!

- All texture binding operations affect the currently active texture unit.
- By default, the 0th texture unit is the active texture unit.
- This can be changed using `glActiveTexture(Glenum textureUnit);`

  where textureUnit is `GL_TEXTURE0, GL_TEXTURE1, ...`
- Once active, a texture can be bound

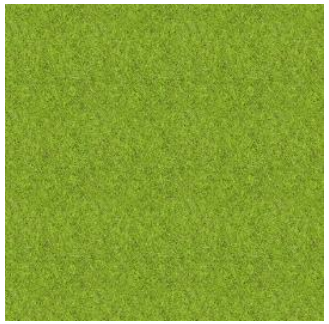
```
glActiveTexture(GL_TEXTURE1);
glBindTexture(m_textureId);
```

  - Note: in the template, this can be done by calling the `Bind(int unit)` method of a CTexture object, unit is the texture unit to use
- Note that any type of texture (1D, 2D, 3D, cube maps) can be loaded into any texture unit

Multi-texturing

- In GLSL, we'll use sampler2Ds to access texels in the different units. On the client side, we must indicate which texture unit to use for each sampler2D.
- For example, suppose in your fragment shader you have four textures that you are applying simultaneously.

```
pTerrainProgram->SetUniform("sampler0", 0);
pTerrainProgram->SetUniform("sampler1", 1);
pTerrainProgram->SetUniform("sampler2", 2);
pTerrainProgram->SetUniform("sampler3", 3);
```



grass
(lowest elevation)

`GL_TEXTURE0`

patchy grass
(low elevation)

`GL_TEXTURE1`

stone
(high elevation)

`GL_TEXTURE2`

snow
(highest elevation)

`GL_TEXTURE3`

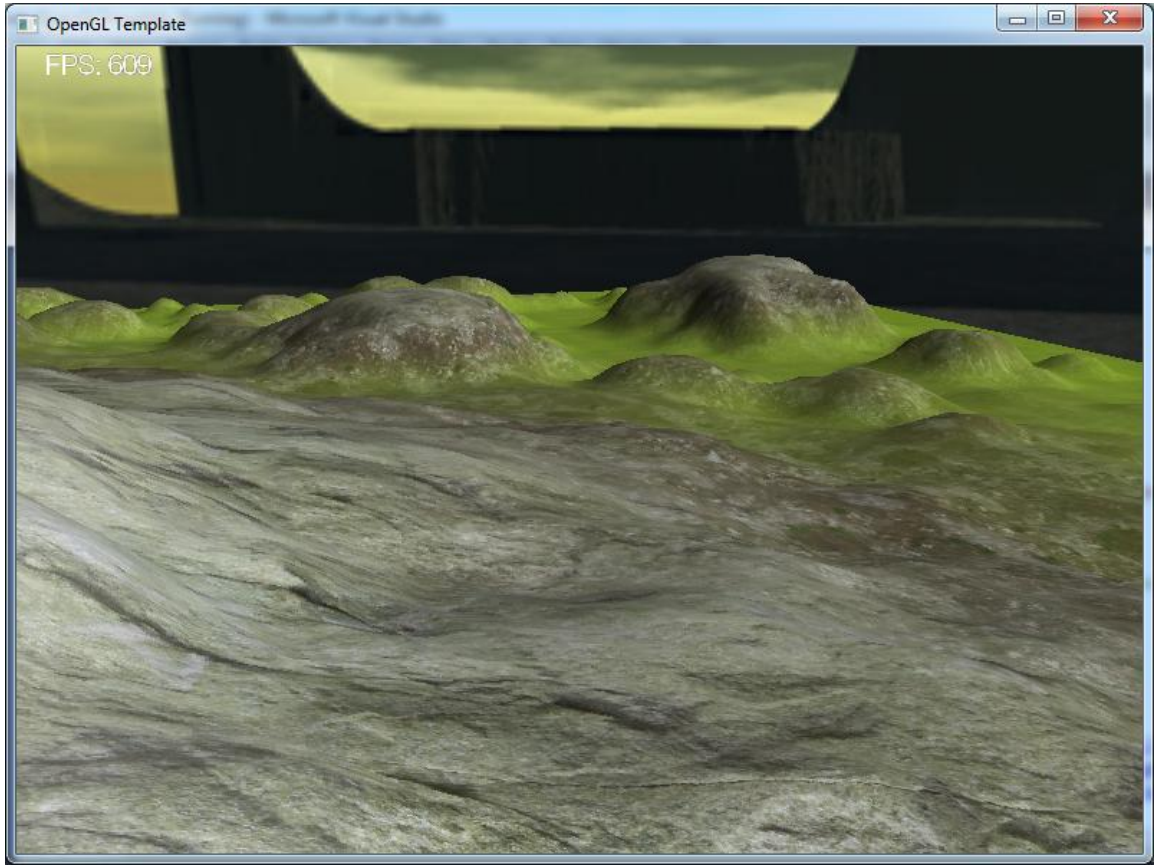
Example code

Fragment shader

```
uniform sampler2D sampler0, sampler1, sampler2, sampler3;
uniform float fMinHeight, fMaxHeight;
in vec3 vWorldPosition, n;
in vec4 p;
...
void main()
{
   vec3 vColour = PhongModel(p, n);
   vec4 vTexColour0 = texture(sampler0, vTexCoord);
   vec4 vTexColour1 = texture(sampler1, vTexCoord);
   vec4 vTexColour2 = texture(sampler2, vTexCoord);
   vec4 vTexColour3 = texture(sampler3, vTexCoord);

   float f = clamp(3*(vWorldPosition.y - fMinHeight) / (fMaxHeight - fMinHeight), 0, 3);
   vec4 vTexColour;
   if (f < 1)
      vTexColour = mix(vTexColour0, vTexColour1, f);
   else if (f < 2)
      vTexColour = mix(vTexColour1, vTexColour2, f - 1.0);
   else
      vTexColour = mix(vTexColour2, vTexColour3, f - 2.0);
   vOutputColour = vTexColour*vec4(vColour, 1.0f);
```

Demo

Terrain multi-textured with four different textures, interpolated by height



Multi-texturing using the template code

1. First, it is necessary to declare multiple textures in your object header file
   - `CTexture m_texture0, m_texture1`

   and then load multiple textures when creating your object:
   - `m_texture0.Load("resources\\textures\\texture0.jpg", true);`
   - `m_texture1.Load("resources\\textures\\texture1.jpg", true);`

2. On the client side, set the sampler2Ds in the fragment shader
   - `pTerrainProgram->SetUniform("sampler0", 0);`
   - `pTerrainProgram->SetUniform("sampler1", 1);`

3. When rendering, bind the textures *before* calling `glDrawArrays`
   - `m_texture0.Bind(0);`
   - `m_texture1.Bind(1);`

4. In the fragment shader, use multiple samplers (`sampler0`, `sampler1`) to access texels in the different textures. As a programmer, it's up to you to combine the texels in an interesting way!

Caution

- It is invalid to have variables of different sampler types pointing to the same texture unit.

C++ (client)

```
pMainProgram->SetUniform("sampler0", 0);
pMainProgram->SetUniform("CubeMapTex", 10);
pMainProgram->SetUniform("sampler1", 2);
```

shader (server)

```
uniform sampler2D sampler0;
uniform sampler2D sampler1;
uniform samplerCube CubeMapTex;
```

✓

C++ (client)

```
pMainProgram->SetUniform("sampler0", 0);
pMainProgram->SetUniform("CubeMapTex", 1);
pMainProgram->SetUniform("sampler1", 1);
```

shader (server)

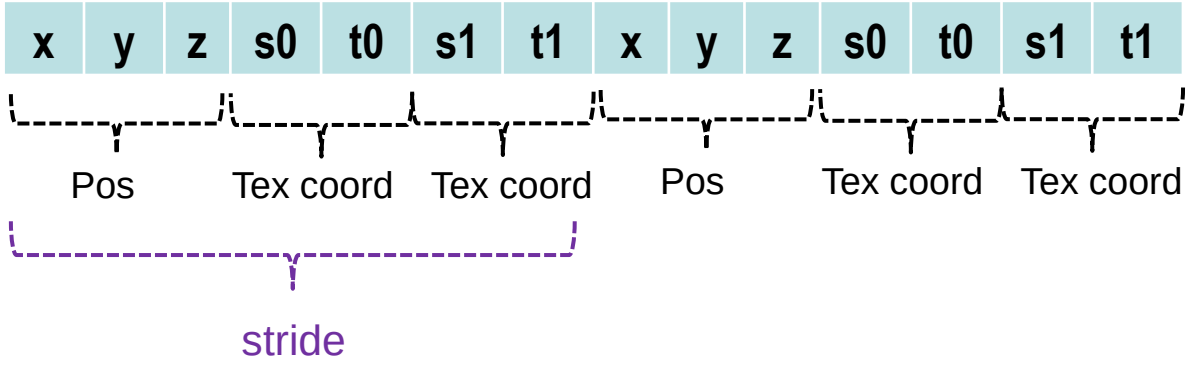
```
uniform sampler2D sampler0;
uniform sampler2D sampler1;
uniform samplerCube CubeMapTex;
```

✗

- The example on the right is invalid since both a sampler2D and samplerCube are pointing to texture unit 1!

Different texture coordinates

- When multi-texturing, it is possible to have different texture coordinates associated with different textures
- In this case, one puts additional texture coordinates in the VBO



Colour buffer

- There are several buffers (colour, depth, stencil) that store pixel-related data, and when combined, form what is called a *framebuffer*.

- The *colour buffer* is a 2D array of pixels that are set during rendering.
- The colour buffer typically resides on the graphics card (server).
- The colour buffer is specified using the pixel format descriptor (Lecture 1). A typical format is 32 bits (8 bits each for R, G, B, A).
- When rendering a frame, the colour buffer is first cleared using

```
glClear(GL_COLOR_BUFFER_BIT);
```

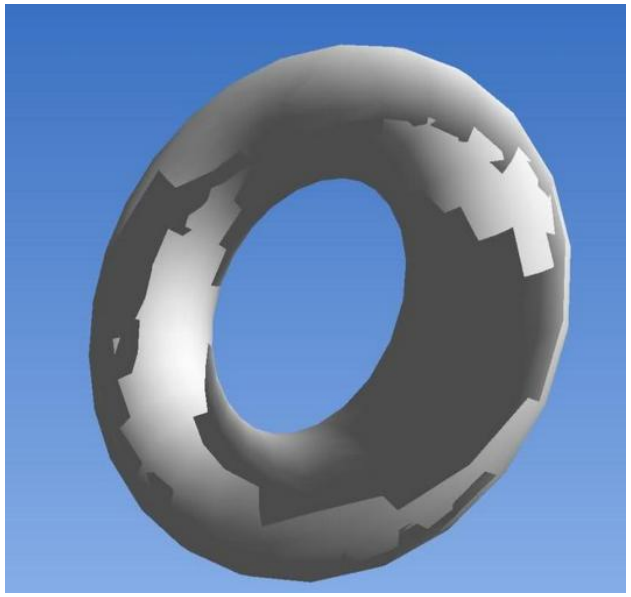
- All pixels in the buffer are set to the colour specified by

```
glClearColor(r, g, b, a);
```

- Note that values in the colour buffer are *clamped*. For example, using floating point colours, each r, g, b, a value is limited to the range [0, 1].

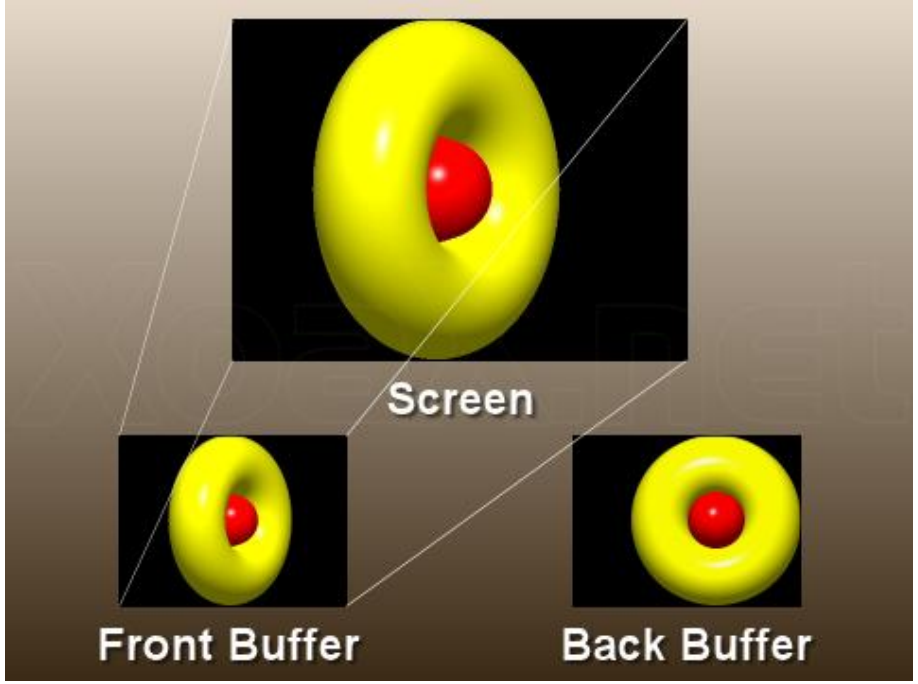
Single buffering

- Artefacts can occur using a single colour buffer
  - The display (screen) is refreshing at a fixed rate, typically 60 Hz.
  - The display hardware will take the contents of the colour buffer to update the screen.
  - If the OpenGL program is updating the colour buffer at the same time the display hardware access the colour buffer, artefacts can appear, especially flickering!



Double buffering

- Double buffering uses a *back buffer* and a *front buffer*
  - The front buffer is the one displayed by the graphics hardware.
  - The back buffer is where the rendering occurs.
  - The two buffers are *swapped* when the rendering of a frame is complete. This can be accomplished using the WGL function `SwapBuffers(hDC);`



http://xoax.net/comp/cpp/opengl/Lesson7.php

Smooth animation using double buffering

- Initialisation: set in the pixel format attributes when the rendering context is created

```
const int iPixelFormatAttribList[] = {
WGL_DRAW_TO_WINDOW_ARB, GL_TRUE,
WGL_SUPPORT_OPENGL_ARB, GL_TRUE,
WGL_DOUBLE_BUFFER_ARB, GL_TRUE,
WGL_ACCELERATION_ARB, WGL_FULL_ACCELERATION_ARB,
WGL_PIXEL_TYPE_ARB, WGL_TYPE_RGBA_ARB,
WGL_COLOR_BITS_ARB, 32,
WGL_DEPTH_BITS_ARB, 24,
WGL_STENCIL_BITS_ARB, 8,
0};
```

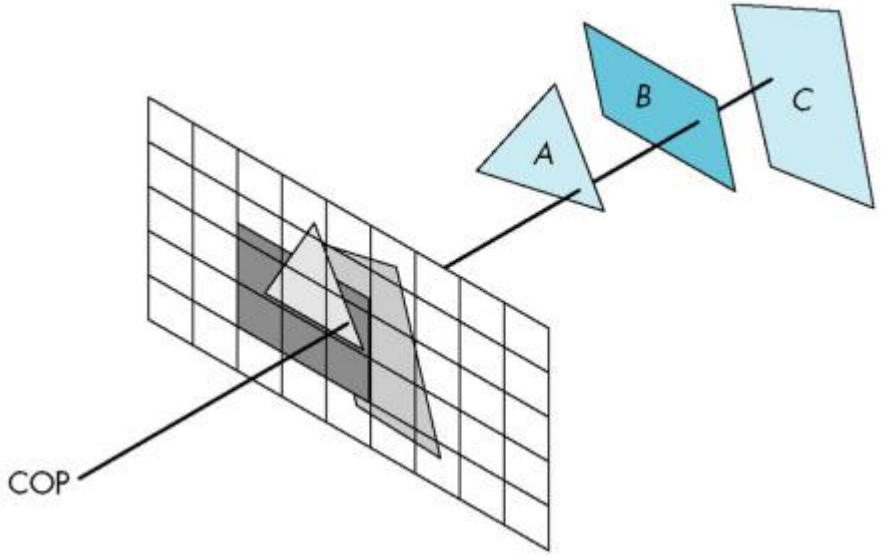
```
glClearColor(0.5f, 0.5f, 0.5f, 1); // Set the background colour
```

- Game::Render() method:

```
glClear (GL_COLOR_BUFFER_BIT); // Clear the back colour buffer

// Draw the scene

SwapBuffers(m_gameWindow.Hdc()); // Swap buffers
```

Depth buffer

- The depth buffer is used to determine visibility, that is, which primitives in the scene are visible to the camera, and which are *occluded* by other primitives. This is an example of *hidden surface removal*.



Example

- Without depth buffering, one sees the horse's mane and tail even though they should be occluded by the horse's body! The order triangles are rendered matters.

Depth test disabled

`glDisable(GL_DEPTH_TEST);`

Depth test enabled

`glEnable(GL_DEPTH_TEST);`

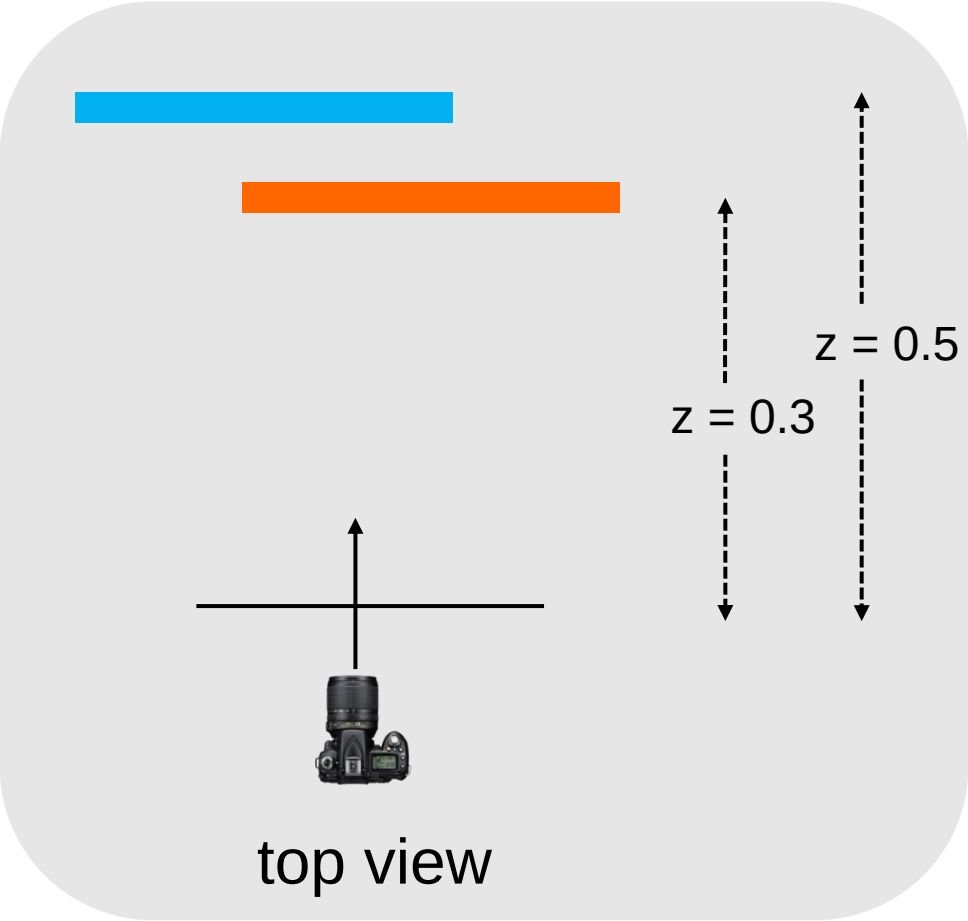
- Alternatively, one could implement the *painter's algorithm,* which draws the primitives from far to near. Implementing this requires depth-sorting the primitives, which is usually inconvenient and less suited to GPU hardware.

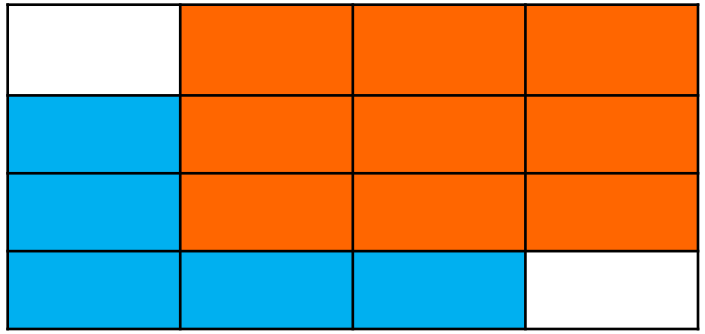
Depth buffer

- The depth buffer, or *z-buffer*, is simply an image that stores, for each pixel, a depth value in the range [0, 1].
- Before rendering, the depth buffer is cleared.
- During rasterisation, a fragment's distance (in eye coordinates) from the camera is compared with the value stored in the depth buffer.
- If the fragment is closer to the camera, its colour is updated along with the depth buffer.
- In code, the algorithm is

```
if ( fragment.z < depthBuffer(x,y).z ) {
    depthBuffer(x,y).z = fragment.z;
    colorBuffer(x,y).color = fragment.color;
}
```

Depth buffer example





correct final image

Depth buffer example

- Initialise the depth buffer

```
glClearDepth(1.0f);
```

1.0	1.0	1.0	1.0
1.0	1.0	1.0	1.0
1.0	1.0	1.0	1.0
1.0	1.0	1.0	1.0

Depth buffer example

- Draw one of the polygons (order does not matter)

1.0	1.0	1.0	1.0
0.5	0.5	0.5	1.0
0.5	0.5	0.5	1.0
0.5	0.5	0.5	1.0

Depth buffer example

- Draw the other polygon

1.0	0.3	0.3	0.3
0.5	0.3	0.3	0.3
0.5	0.3	0.3	0.3
0.5	0.5	0.5	1.0

- Advantage: order-independent rendering
- Disadvantage: may write the same pixel multiple times

Depth buffering in OpenGL

- Initialisation: set in the pixel format attributes when the rendering context is created

```
const int iPixelFormatAttribList[] = {
WGL_DRAW_TO_WINDOW_ARB, GL_TRUE,
WGL_SUPPORT_OPENGL_ARB, GL_TRUE,
WGL_DOUBLE_BUFFER_ARB, GL_TRUE,
WGL_ACCELERATION_ARB, WGL_FULL_ACCELERATION_ARB,
WGL_PIXEL_TYPE_ARB, WGL_TYPE_RGBA_ARB,
WGL_COLOR_BITS_ARB, 32,
WGL_DEPTH_BITS_ARB, 24,
WGL_STENCIL_BITS_ARB, 8,
0};
```

```
glEnable(GL_DEPTH_TEST); // Enable depth test (z-buffering)
```

- Game::Render() method:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // Clear buffers

// Draw the scene

SwapBuffers(rGameWindow.Hdc()); // Swap buffers
```

Non-linear values

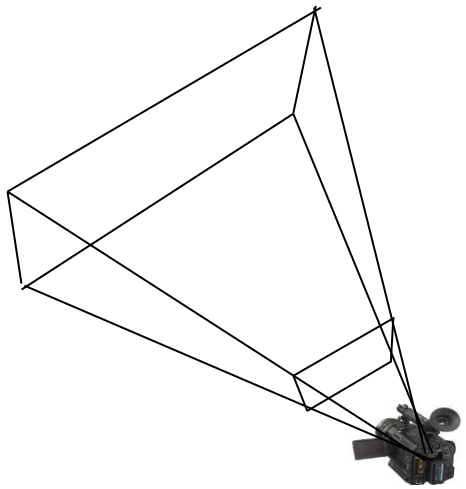
- The value stored by the hardware in the depth buffer is a non-linear mapping of the actual depth from the camera based on the near and far clipping planes
- This is done so that objects closer to the camera have more depth resolution
- The mapping is of the form
  z_buffer_value ~ a - b / z

  where

  a = zFar / (zFar - zNear)

  b = zFar * zNear / (zFar – zNear)

  z = distance from camera to object
- z-resolution inversely proportional to distance
- Tip: set zNear to a value > 0



http://www.sjbaker.org/steve/omniv/love_your_z_buffer.html

z-fighting

- z-fighting is a rendering artefact that occurs when primitives have similar values in the depth buffer. It often happens for co-planar polygons or when the depth buffer doesn't have enough resolution.
- Example: co-planar polygons. Neither polygon is closer to the camera; due to quantisation issues, either polygon may "win" the depth buffer test as both are "fighting" to colour the screen pixels.

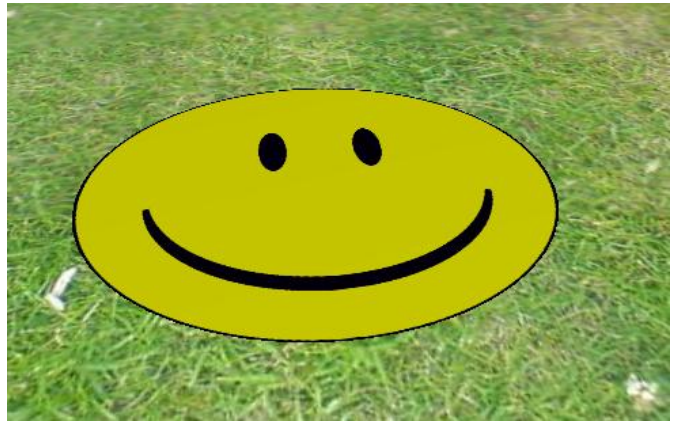
Disk rendered on the same plane as the ground

Resolving issues with the depth buffer

- It may be a good idea to allocate more bits to the depth buffer. This will allow more discrete distances to be stored. Usually the depth buffer is 24 bits.
- If coplanar polygons *must* be rendered, one may wish to use a *polygon offset*, which offsets the z-value (in eye coordinates) for the polygon being rendered. Offsets can be positive or negative (negative will move the object closer to the camera) and are computed before depth buffering.
- Polygon offsets can by enabling GL_POLYGON_OFFSET_FILL

```
glPolygonOffset(GLfloat factor, GLfloat units);
```

- The offset is *factor* × dz + *units* × r, where dz is the change in depth relative to the screen area, and r is the smallest resolvable offset in the depth buffer.



Multi-pass rendering

- Rendering is not complete until we call `SwapBuffers();`
- In some cases, we may wish to render the scene with multiple passes
  - Render some or all the scene once
  - Change some parameters
  - Render some or all the scene again

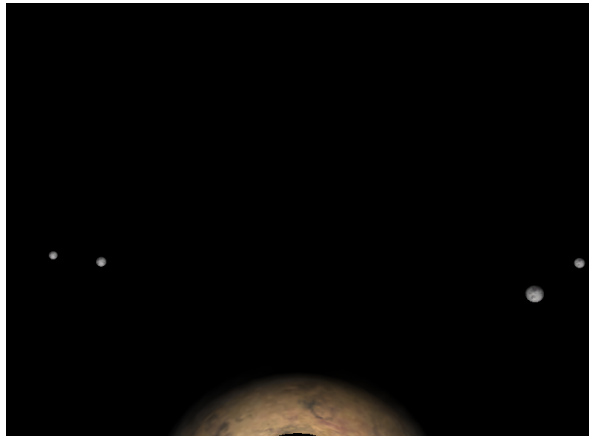
Greg Norman Ultimate Challenge Golf

Rendering the scene twice:
1. Top view
2. Behind player

Mirror-like reflecting ground plane

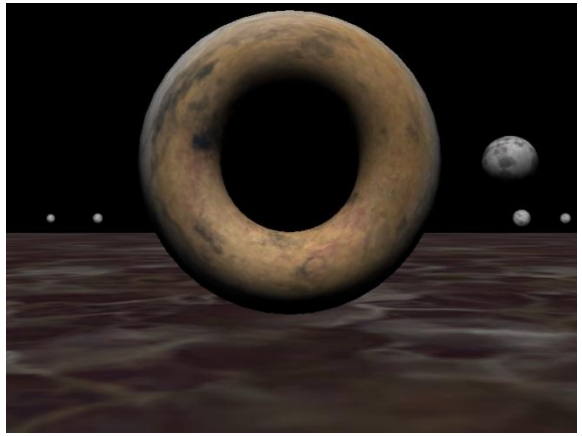
This also requires multi-pass rendering

- Pass 1: Render the scene to be reflected upside down.
- Pass 2: Draw the ground with alpha blending on, and the scene to be reflected right side up with alpha blending off



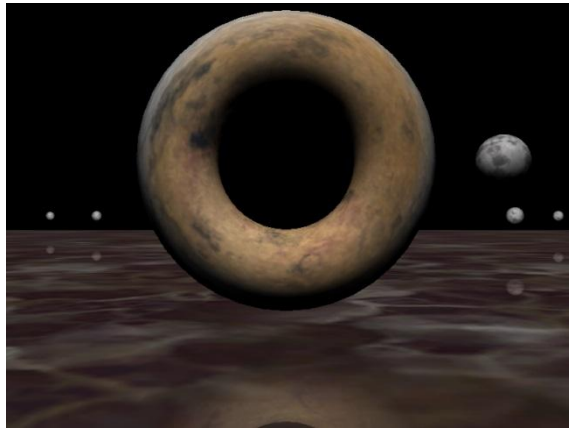
pass 1

"+"



pass 2

=

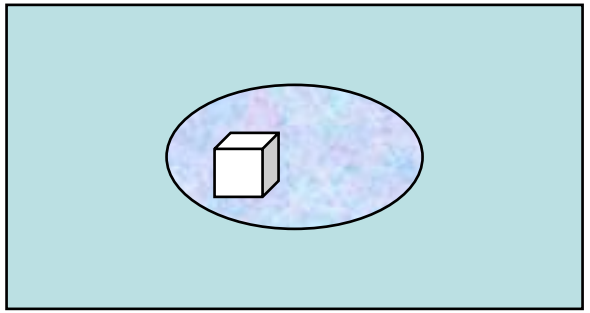


result

Sphereworld demo – Chapter 5 of the OpenGL Superbible (5th edition)

Stencil buffer

- Stenciling, like depth-buffering, enables and disables drawing on a per-fragment basis.
  - A stencil is first created, usually by rendering primitives
  - Subsequent rendering uses the stencil to mask out portions of the screen
  - Stenciling is often used in multi-pass rendering



http://downtoearthstyle.blogspot.com/

⇨ Similar to how a painter might limit drawing to a defined region using a stencil

Stencil buffer

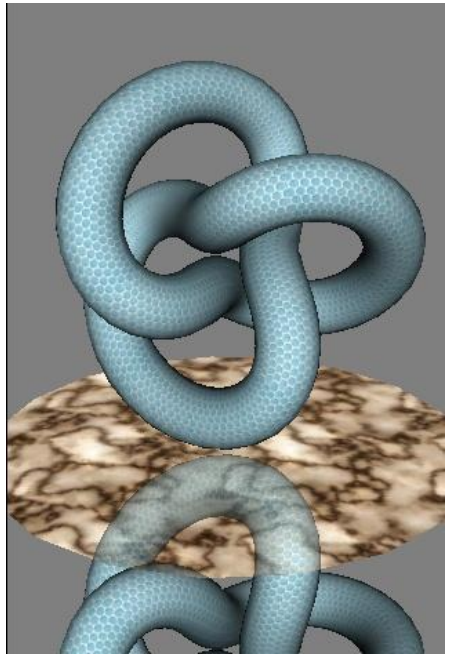
- The stencil buffer is stored as an additional buffer on the graphics card
  - Storage is associated with each framebuffer pixel

- If enabled, the stencil buffer can be written to during rendering
- When rendering other geometry, each fragment can be tested against the stencil buffer
  - Fragments that pass the test are rendered
  - Fragments that fail the test are discarded

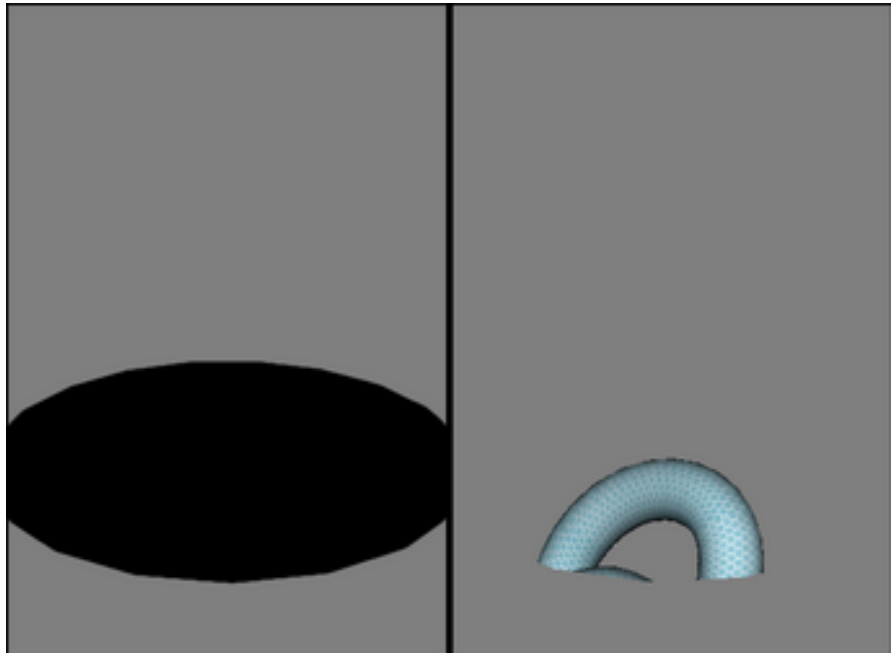
⇨ Example: create a mask in the stencil buffer and only draw objects in the masked area

Example (planar reflection)

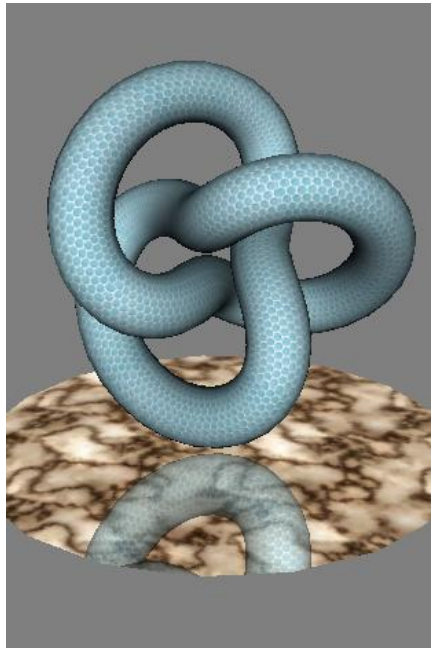
- Mirror-like reflection off the ground plane, by multi-pass rendering



Problem: want to limit reflection to marbled floor



Stencil and stenciled reflection



result

Stencil buffer

- Stenciling is an OpenGL state, which can be enabled/disabled:

```
glEnable(GL_STENCIL_TEST);
```

- The value stored at a pixel in the stencil buffer may be binary (single bit) or multiple bits. In this way, the stencil buffer can effectively store multiple stencils.
- The number of bits in the stencil buffer set by the pixel format attributes when the rendering context is created:

```
const int iPixelFormatAttribList[] = {
WGL_DRAW_TO_WINDOW_ARB, GL_TRUE,
WGL_SUPPORT_OPENGL_ARB, GL_TRUE,
WGL_DOUBLE_BUFFER_ARB, GL_TRUE,
WGL_ACCELERATION_ARB, WGL_FULL_ACCELERATION_ARB,
WGL_PIXEL_TYPE_ARB, WGL_TYPE_RGBA_ARB,
WGL_COLOR_BITS_ARB, 32,
WGL_DEPTH_BITS_ARB, 24,
WGL_STENCIL_BITS_ARB, 8,
0};
```

Programming the stencil buffer

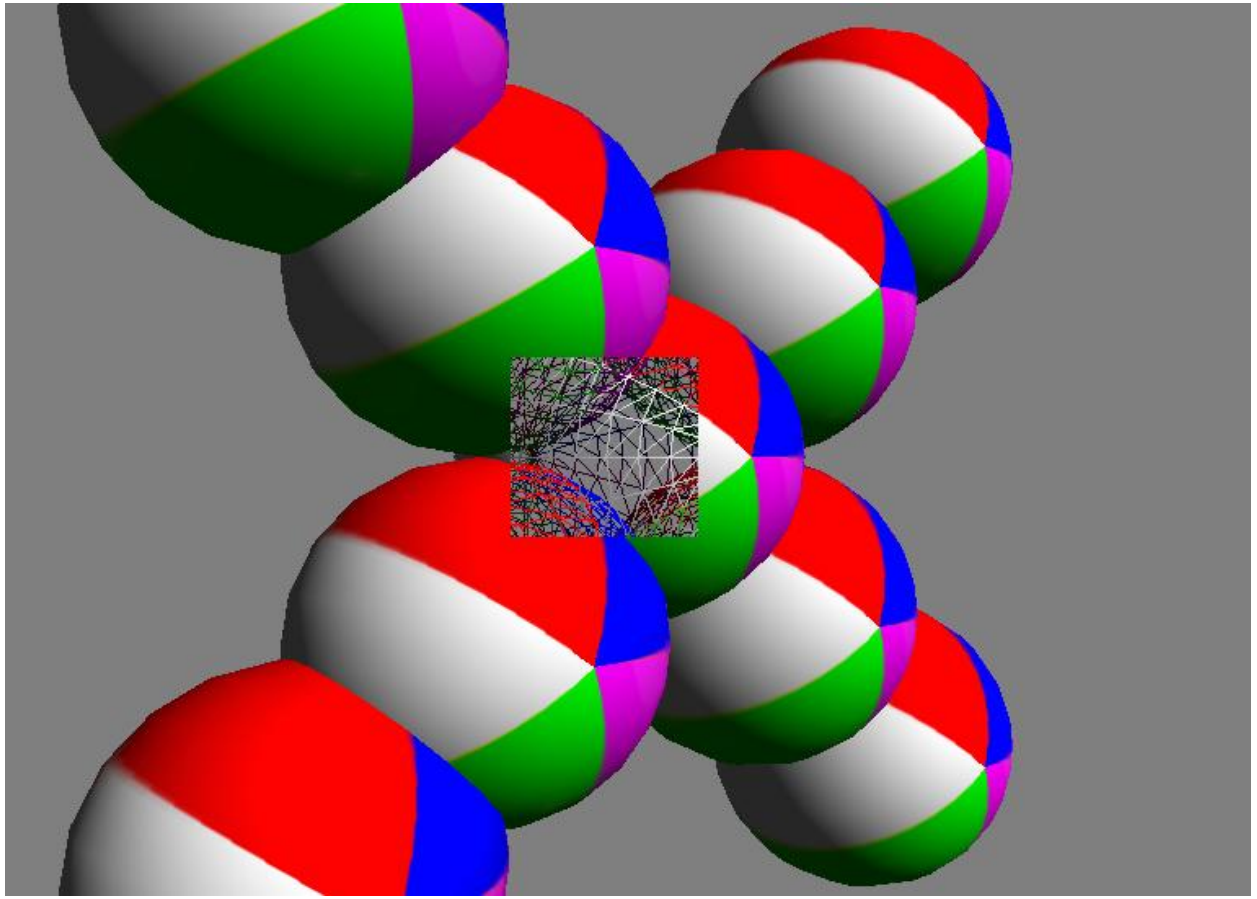
- `glStencilFunc(` *`func, ref, bitmask`* `)`
  - Controls how the stencil buffer determines *if a pixel is writeable*
  - Compares the value in the buffer with `ref` using `func`
  - `func` is one of standard comparison functions, such as
    - `GL_ALWAYS:` test always passes
    - `GL_LESS:` passes if (*ref* & *bitmask*) < (*stencil* & *bitmask*)
    - `GL_LEQUAL:` passes if (*ref* & *bitmask*) <= (*stencil* & *bitmask*)
    - `Others(GL_GREATER, GL_GEQUAL, GL_EQUAL, GL_NOTEQUAL, GL_ALWAYS)`
  - When using multiple levels of stencil, *`bitmask`* allows the desired level(s) to be used
  - Often, we have a binary stencil and set *`bitmask`* to 1.

⇨ If the stencil test fails, no change is made to the fragment's colour or depth buffers

Programming the stencil buffer

- `glStencilOp( sfail, zfail, zpass )`
- *Allows changes to the stencil buffer* when the stencil test fails
- *`sfail`* can be one of several options:
  - `GL_KEEP`: keep the current value of the stencil buffer
  - `GL_ZERO`: set the stencil buffer to zero at pixels that fail the test
  - `GL_REPLACE`: sets the stencil buffer to `ref` from `glStencilFunc`
  - `(others)`
- For fragments that pass the stencil buffer test, they are then subjected to a depth buffer test. *`zfail, zpass`*: describe how to update the stencil buffer when the depth test fails or passes, respectively.

Demo: solid and wireframe



Programming example

Initialisation:

```
glEnable( GL_STENCIL_TEST );
glClearStencil( 0x0 );
```

Render():

```
glClear(GL_COLOR_BUFFER_BIT |  GL_DEPTH_BUFFER_BIT |
    GL_STENCIL_BUFFER_BIT);
glStencilFunc( GL_ALWAYS, 1, 1 );
glStencilOp( GL_REPLACE, GL_REPLACE, GL_REPLACE );
```

// Render stencil mask -- regardless of the test, we replace the value in the stencil buffer with 0x1.

Programming example

Render(), continued:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glStencilFunc(GL_EQUAL, 1, 1);
glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
// Render scene inside stencil

glStencilFunc(GL_NOTEQUAL, 1, 1);
glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
// Render scene outside stencil


SwapBuffers(rGameWindow.Hdc());
```

We've completed basic computer graphics!

One could make a pretty compelling 3D game with what we've covered already

Skybox

Ambient + diffuse reflection

Mesh with different textures

2D overlays (orthographic projection)

Directional light source

Alpha-blended texture map shadows

Perspective projection

Going beyond the window

- When one thinks of 3D rendering, it is common to think of 3D graphics rendered to the screen.
- When creating a rendering context, a default framebuffer is constructed, and attached to the current window. Rendering is performed in this framebuffer.
- However, in some cases we may be interested in rendering to memory (offscreen) instead.
- Offscreen rendering can be useful for numerous effects, like
  - Mirrors, CCTV screens
  - Image processing (edge detection, blur)
  - Bloom
  - Shadows
  - Deferred rendering
  - Dynamic cube mapping
  - (others)

Framebuffer object (FBO)

- Modern OpenGL encapsulates the state required for drawing to a framebuffer into an object called a *framebuffer object* (FBO).
- An FBO is a container that holds an object in memory that can be rendered to.
- A programmer can create an FBO for offscreen rendering. An FBO may be configured differently than the window tied to the rendering context. For example, the FBO may have a different size.
- Good details in the SuperBible

Creating and binding an FBO

- Creating FBOs can be achieved using the call

```
void glGenFramebuffers(GLsizei n, GLuint *ids);
```

  where `n` is the number of FBOs to be created, and `ids` are FBO ids

- To bind an FBO to make it active, call

```
void glBindFramebuffer(GLenum target, GLuint id);
```

  where target is (typically) `GL_FRAMEBUFFER` and `id` is the FBO id

- To delete FBOs, call

```
void glDeleteFramebuffers(GLsizei n, GLuint *ids);
```

Framebuffer attachments

- Remember, an FBO is a container. Now that we can create and bind an FBO, we need something to put in the container! There are three possible types of attachments supported:
  - Texture (colour buffer)
  - Depth buffer
  - Stencil buffer

  } Needed for rendering to texture

```
// Create a colour texture for the FBO
glGenTextures(1, &m_uiColourTexture);
glBindTexture(GL_TEXTURE_2D, m_uiColourTexture);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, a_iWidth, a_iHeight, 0, GL_BGRA,
GL_UNSIGNED_BYTE, NULL);

// Create a depth texture for the FBO
glGenTextures(1, &m_uiDepthTexture);
glBindTexture(GL_TEXTURE_2D, m_uiDepthTexture);
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT24, a_iWidth, a_iHeight, 0,
GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, NULL);
```

Framebuffer attachments

- Attach the colour and depth textures to the FBO:

```
glFramebufferTexture(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
                     m_uiColourTexture, 0);
glFramebufferTexture(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT,
                     m_uiDepthTexture, 0);
```

- Tell OpenGL that we want to draw into the framebuffer's colour attachment:

```
static const GLenum draw_buffers[] = {GL_COLOR_ATTACHMENT0};
glDrawBuffers(1, draw_buffers);
```

- Check that the framebuffer is complete:

```
GLenum returnCode = glCheckFramebufferStatus(GL_FRAMEBUFFER);
```

Helper class

- On Moodle, there is a helper class called `CFrameBufferObject`
- It handles all these operations for you!

```
CFramebuffer *m_pFBO;

m_pFBO->CreateFramebuffer(width, height);
```

- Note: to unbind a framebuffer, simply call

```
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Rendering to texture

1. Create a FBO of a certain width and height in `Game::Initialise()`
2. Set the viewport and perspective projection matrix based on the FBO size
3. Bind the FBO and render the scene a first time

```
m_pFBO->Bind(); // Render to texture
RenderScene(0); // pass 0 – don't render TV screen
```

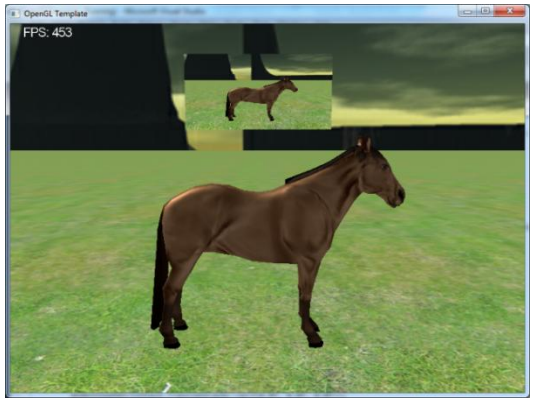
4. Reset the viewport and perspective projection matrix based on the window size
5. Go back to rendering to the default framebuffer, and render the scene again.

```
glBindFramebuffer(GL_FRAMEBUFFER, 0); // Render to window
RenderScene(1); // pass 1 – render scene including TV screen
```

6. Bind the texture in the FBO and render an object not present in first pass (e.g., TV screen)

```
m_pFBO->BindTexture(0); // draw TV screen using the
framebuffer texture
```

7. Swap buffers



Helper class and the template

- Simplest approach: Create an FBO that has the same size as the window. Then you don't have to modify the projection matrix.
  - Put the FBO helper class into the template and instantiate an `m_pFBO` object.
  - In Game::Initialise(), call `m_pFBO->Create(width, height);`
  - Rename `Game::Render()` as `Game::RenderScene(int pass)` and update Game.h header
  - Create a new `Game::Render()` method and update Game.h header
  - Move the calls to `DisplayFrameRate()` and `SwapBuffers(m_gameWindow.Hdc())` to the new `Game::Render()` method.
  - In `Game::Render()`,
    - Bind the FBO to render to texture
    - Call `RenderScene(0)`, and render the scene as usual
    - Unbind the FBO to render to the window `glBindFramebuffer(GL_FRAMEBUFFER, 0);`
    - Call `RenderScene(1)`, and render the scene as usual, but include an additional object (like a TV screen, cube, etc.) textured with the texture from the FBO. Recall to bind the FBO texture, one calls `m_pFBO->BindTexture(0);`

Portal

Image processing

- FBOs are also very useful for image processing, which will apply some sort of transformation to the image pixels before they are displayed.
- Common image processing examples include
  - Edge detection
  - Blur
  - Colour correction, tone mapping, gamma correction, vignetting
  - Bloom
  - (others)
- Often these are used as post-processing effects

⇨ We are going to perform these operations in hardware using shaders.

Image processing

- Image processing operations typically require that, at a given pixel, we know the value at an adjacent pixel.
  - We will store an image as a texture.
  - Using a sampler object, we can sample different texels in the texture.

```
uniform sampler2D sampler0;
uniform int width;
uniform int height;
void main()
{
   float dx = 1.0 / (float)width;
   float dy = 1.0 / (float)height;
   vec4 thisTexel = texture(sampler0, vTexCoord);
   vec4 leftTexel = texture(sampler0, vTexCoord + vec2(-dx, 0.0));
   vec4 rightTexel = texture(sampler0, vTexCoord + vec2(dx, 0.0));
   vec4 aboveTexel = texture(sampler0, vTexCoord + vec2(0.0, dy));
   vec4 belowTexel = texture(sampler0, vTexCoord + vec2(0.0, -dy));
...
```

  - We can then apply some mathematical operations to the texels.
  - And save the output to:
    - Another texture in an FBO
    - The colour buffer in the default framebuffer

2D filtering

- Many image processing operations are implemented using a *filter (or kernel)*, which is applied to each pixel in an image.
- In 2D filtering, the filter is simply a matrix of values that describe how to transform a pixel by replacing it with the sum of products between the values of nearby pixels and the filter weights.
- For example, consider the following 3x3 filter, and a set of pixel values (grayscale intensities)

	1	0	1		25	26	27
Filter:	0	2	0	Pixels:	28	29	30
	1	0	1		31	32	33

- To apply the filter, we simply multiply corresponding elements together and sum the results.
- In this case, the value would be (25 + 27 + 2*29 + 31 + 33) = 174

2D filtering

- Typically the image is much larger than the filter.
- The filter is moved through the image to produce filtered outputs, which are stored in a separate buffer.
- The filter can be any size (NxM matrix)

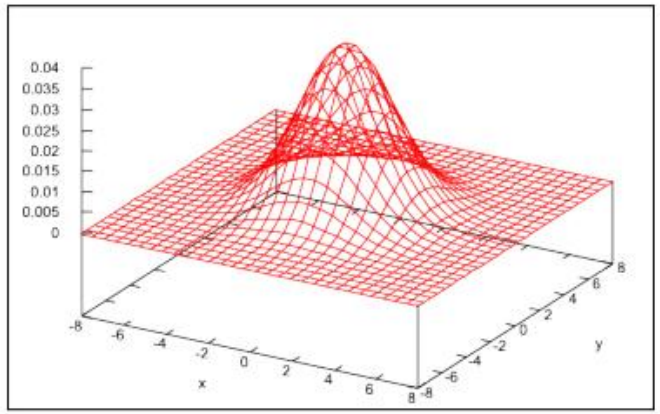
23	22	11	17	23
12	15	24	45	13
34	35	34	34	14
25	44	17	23	15
33	23	14	11	13

$$f[x,y] * g[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2] \cdot g[x-n_1, y-n_2]$$

http://graphics.stanford.edu/courses/cs178/applets/convolution.html

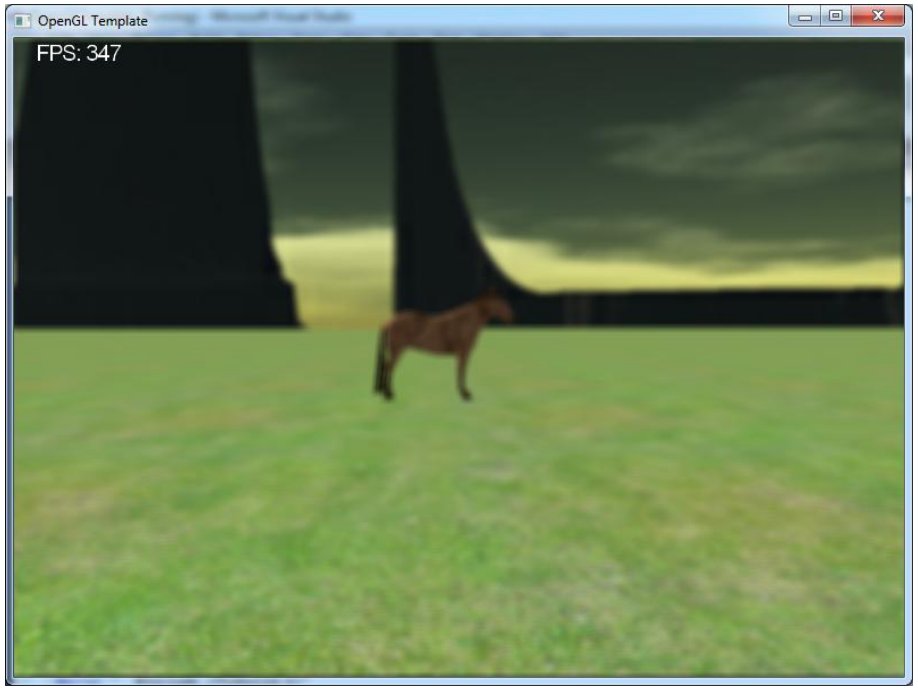
Blur

- Blur is another image processing operation that can be computed with a 2D filter. These filters are lowpass filters, since they pass the low frequencies (signal that changes slowly) but diminish the higher frequencies (edges).
- The idea is to mix the colour of a pixel with that of its neighbours using a weighted sum.
- There are many types of lowpass filter kernels. Popular ones include:
  - Moving average, also called a *box* filter. This just averages the pixels in a neighbourhood around a pixel.
  - Gaussian filter. This applies a weighting based on a 2D Gaussian function. The neighbours are weighted so that ones farther away contribute less.
  - (others)



Blur

- Pass 1: Render the scene to texture using an FBO
- Pass 2: Render a full-screen quad using the texture from pass 1 to the default framebuffer. In the fragment shader,
  - Set the fragment colour to the average of the pixels in an NxN window around the current fragment



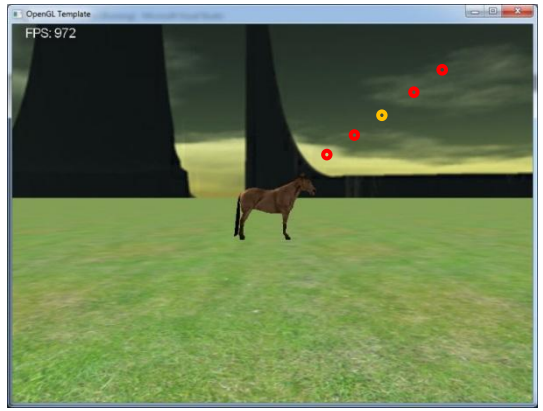
Blur (exercise): 2D moving average code

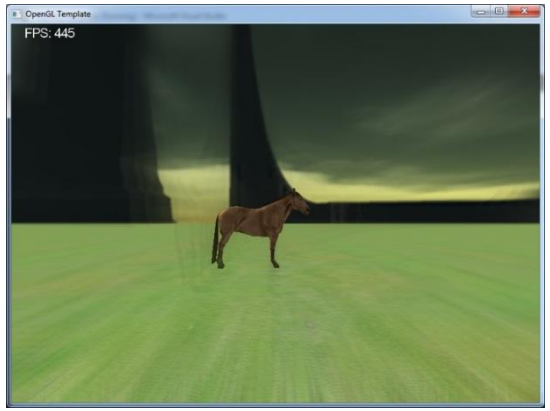
```
#version 400
in vec2 vTexCoord;
out vec4 vOutputColour;
uniform sampler2D sampler0;
uniform vec4 vColour;
uniform int width, height;
void main()
{
   float dx = 1.0 / float(width);
   float dy = 1.0 / float(height);

   int halfSize = 3;
   vec4 avg = vec4(0);
   for (int y = -halfSize; y <= halfSize; y++) {
      for (int x = -halfSize; x <= halfSize; x++) {
         avg += texture(sampler0, vTexCoord+vec2(x*dx, y*dy));
       }
   }
   int N = (2*halfSize+1);
   avg /= N*N;
   vOutputColour = avg;
}
```

fragment shader
(second pass)

Radial blur

- Pass 1: Render the scene to texture using an FBO
- Pass 2: Render a full-screen quad using the texture from pass 1 to the default framebuffer. In the fragment shader,
  - Form a vector from the centre of the screen (texture coordinate $[0.5, 0.5]^T$) and the current texture coordinate.
  - Sample the image along this vector, forming a blurred colour.
  - Use `mix`, based on distance, so that pixels farther from the centre get more blurring
- Tutorial: http://stackoverflow.com/questions/4579020/how-do-i-use-a-glsl-shader-to-apply-a-radial-blur-to-an-entire-scene
- *Note: `texture2D()` is deprecated -- use `texture()` instead.*

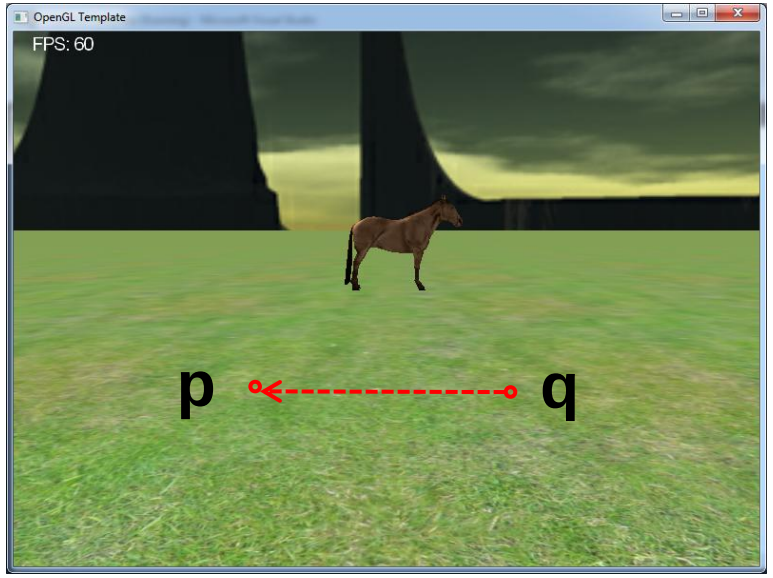




Motion blur

- Motion blur is also a screen space technique.
- It works by determining, for a current fragment **p**, where it was located on a previous frame (let's call this **q**).
- A blur vector is formed between **p** and **q**, and colours are averaged.

http://john-chapman-graphics.blogspot.co.uk/2013/01/what-is-motion-blur-motion-pictures-are.html



Motion blur

- Pass 1: Render the scene to texture using an FBO
- Pass 2: Render a full-screen quad using the texture from pass 1 to the default framebuffer. In the fragment shader,
  - Retrieve the distance to the point from the depth buffer, forming **p**
  - Unproject to local coordinates using the inverse projection matrix and inverse modelview matrix, forming **p3d**.
  - Project **p3d** using the *previous* frame's modelview / projection matrix and shift to the range [0, 1], forming **q**.
  - Blur the image along the vector between **p** and **q**.

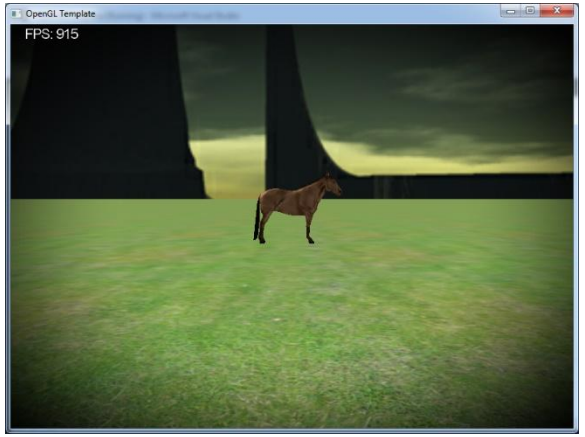
fragment shader
(second pass)

```
#version 400
in vec2 vTexCoord;
out vec4 vOutputColour;
uniform sampler2D sampler0; // used to access the rendered image from pass 1
uniform sampler2D sampler1; // used to access the depth buffer
uniform mat4 invMVP;
uniform mat4 prevMVP;

void main()
{
  // Unproject the point to 3D using the current modelview and projection matrix
  float p_depth = texture(sampler1, vTexCoord).r;
  vec2 p_ndc =  vTexCoord.xy * 2.0f - 1.0f;
  vec4 p3d = invMVP * vec4(p_ndc, p_depth, 1.0);
  p3d = p3d / p3d.w;

  // Project the point using the previous frame's modelview and projection matrix
  vec4 q = prevMVP * p3d;
  q.xyz /= q.w; // normalised device coordinates
  q.xy = q.xy * 0.5 + vec2(0.5); // texture coordinates

  // Construct the blur vector (scaled for extra blur)
  vec2 blurVec = (q.xy - vTexCoord)*5.0f;
  vec4 result = texture(sampler0, vTexCoord);
  int nSamples = 11;
  for (int i = 1; i < nSamples; i++) {
     vec2 offset = blurVec * (float(i) / float(nSamples - 1) - 0.5);
     result += texture(sampler0, vTexCoord + offset);
  }
  result /= float(nSamples);
  vOutputColour = result;
}
```

Vignetting

- Vignetting is a reduction of an image's brightness or saturation at the periphery compared to the image centre.
- Pass 1: Render the scene to texture using an FBO
- Pass 2: Render a full-screen quad using the texture from pass 1 to the default framebuffer. In the fragment shader,
  - Compute the distance from the centre of the screen.
  - Darken the fragment based on this distance.
  - Code below inspired from https://github.com/mattdesl/lwjgl-basics/wiki/ShaderLesson3

```
// some things above omitted
void main()
{
   vec4 texColour = texture(sampler0, vTexCoord);
   vec2 vectorFromCentre = vTexCoord.xy - vec2(0.5);
   float vignette = 1 - smoothstep(0.3, 0.75,
                     length(vectorFromCentre));
   texColour.rgb *= vignette;
   vOutputColour = texColour;
}
```



Bloom

- Bloom is a visual effect where the bright parts of an image seem to have fringes that extend beyond their boundaries.
- Sources of bright light “bleed” into other areas of the image.

Elephant’s Dream

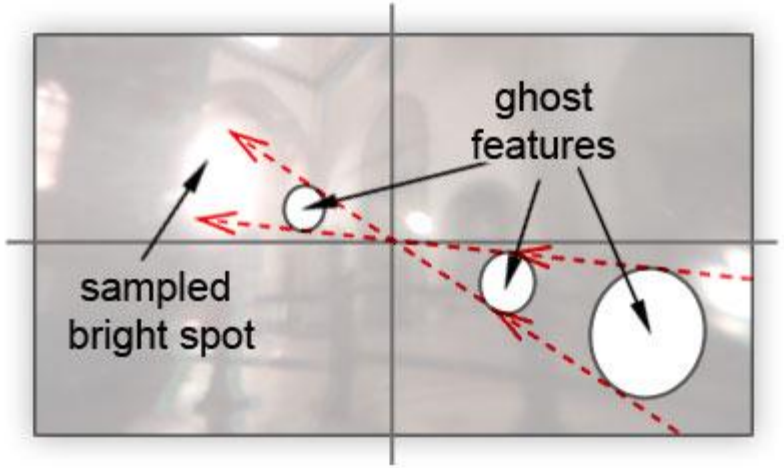
Bloom

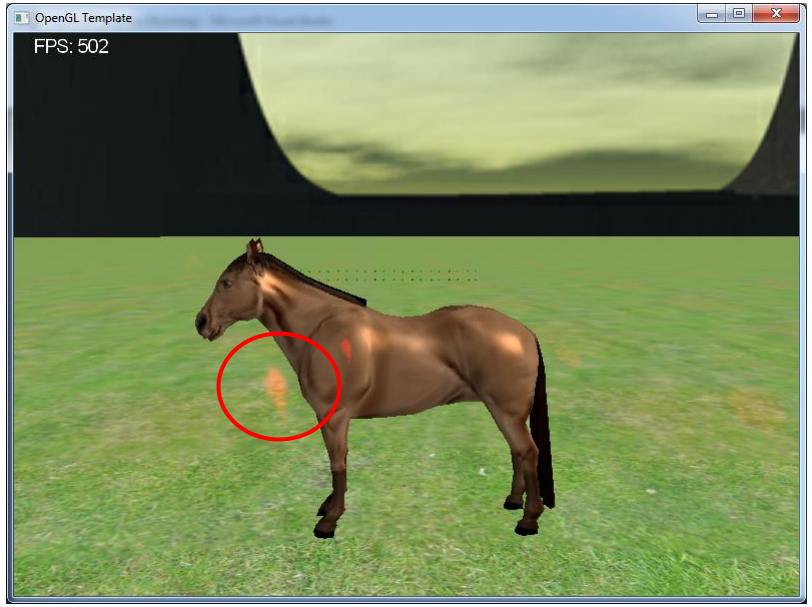
- We can mimic this effect by determining which parts of the image are bright.
- Then, blur those bright parts, and add the blurred result back into the image.
- [recipe courtesy of Greg Slabaugh]
  - Pass 1: Render the scene to texture using FBO1.
  - Pass 2: Render a full-screen quad using the texture from pass 1 to texture using FBO2. In the fragment shader,
    - Set fragments with brightness below a threshold to zero.
  - Pass 3: Render a full-screen quad to the default framebuffer. In the fragment shader,
    - Apply a blur using the texture in FBO2.
    - Add it to the original image using the texture in FBO1.

Lens flare

- Lens flare is an optical effect resulting from light scattered in a lens.
  - Pass 1: Render the scene to texture using FBO1.
  - Pass 2: Extract the bright parts of the image and put into FBO2.
  - Pass 3: Mirror bright parts by pivoting around the image centre forming “ghosts”, and add to original rendered image.
  - *[more]*

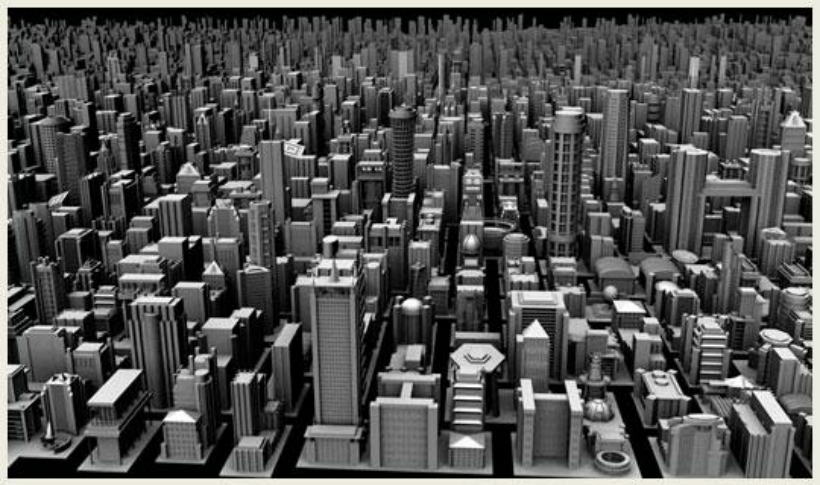
http://john-chapman-graphics.blogspot.co.uk/2013/02/pseudo-lens-flare.html





Procedural methods

- Procedural methods apply an algorithm (procedure) to synthesize data. Wide applicability
  - Terrain
  - Textures
  - Special effects
  - Trees and foliage
  - Cities



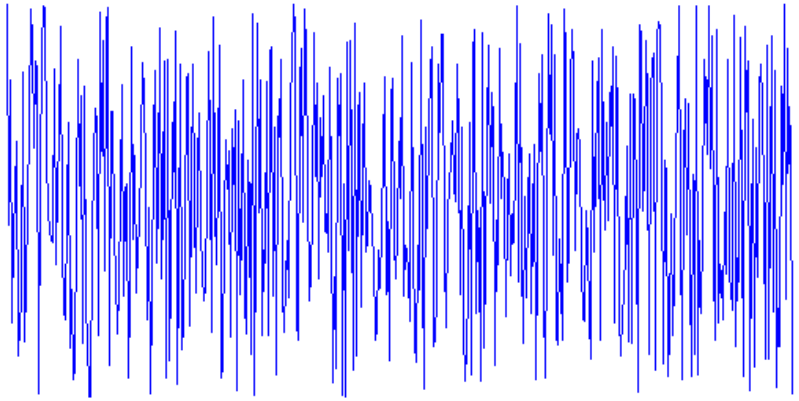
http://www.speedtree.com/gallery/index.php?Page=42

http://lesterbanks.com/2010/02/a-few-building-city-generation-tools-for-maya/

White noise

- Using a pseudo-random number generator, it is easy to make noise in any number of dimensions (e.g., 1D, 2D, 3D, and 4D).

```
float vector1D[500];
for (int i = 0; i < 500; i++)
    vector1D[i] = rand() / RAND_MAX;
```



- If one were to take the Fourier transform of such a signal, one would see that it contains all frequencies (low, medium, high) in equal amounts.
- White light has all frequencies in the visible electromagnetic spectrum. By analogy, noise that contains all frequencies is called “white”.
- One could try to model terrain using white noise. However, the signal would vary too much and the terrain would not look realistic – just random.

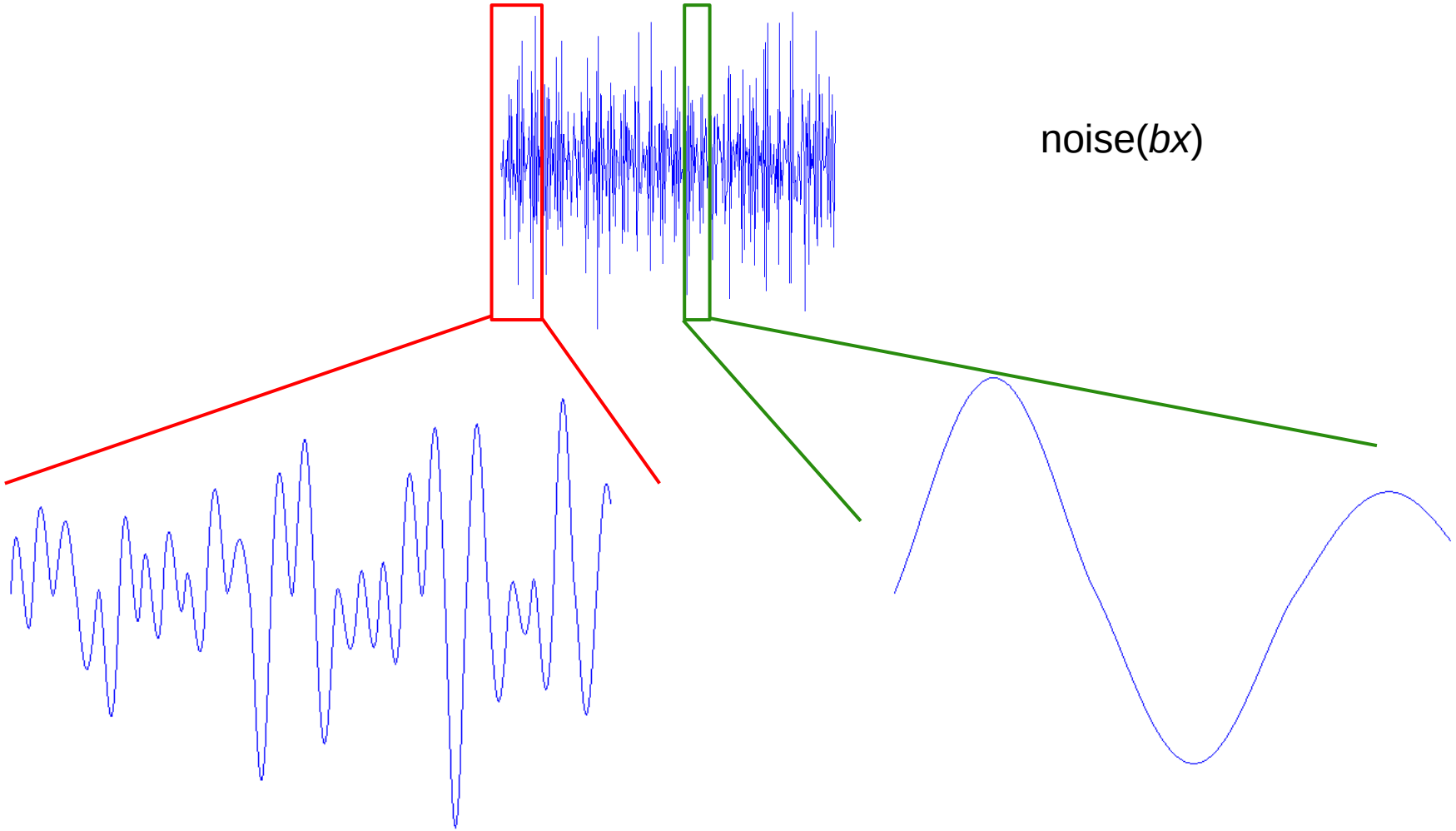
Perlin noise

- Perlin noise is *controlled noise* with configurable frequencies
  - It is also computed using pseudo-random number generators
  - It uses interpolation to band-limit frequencies of the noise
  - It also allows octaves (i.e., powers of a given frequency), which are added to produce the final noise signal
- Many effects in computer games are based on Perlin noise. Some examples include, but are not limited to
  - 2D texture synthesis for rock, marble
  - 2D heightmaps for terrain generation: mountains, valleys
  - 3D animated textures: smoke, fire, clouds

Sample heightmap generated using Terragen

Interpolated noise

- One can select a set of samples based on a parameter *b*
- Then, one can smoothly interpolate the samples



Perlin noise

- Perlin noise sums this equation over several different *octaves*, which are powers of the frequency
- The complete equation is

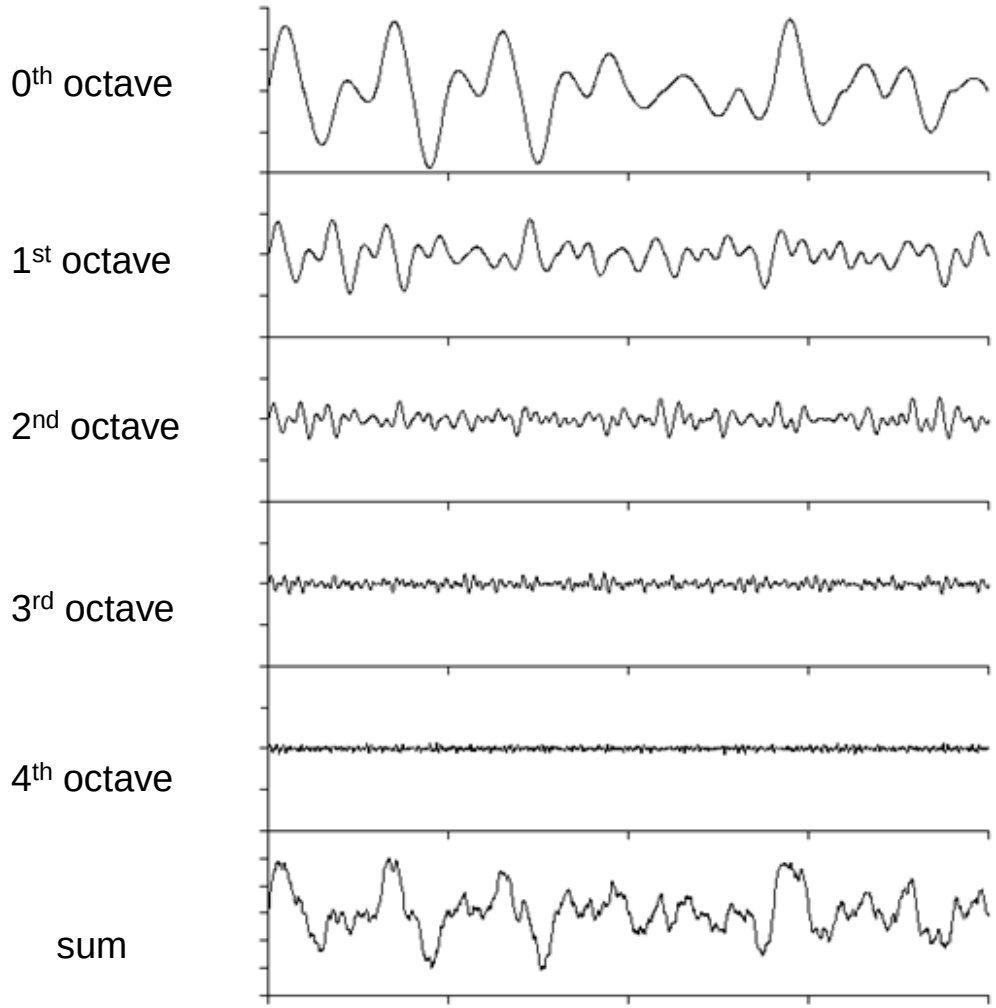
$$\mathrm{Perlin}(x, a, b, N) = \sum_{i=0}^{N-1} \frac{\mathrm{noise}(b^i x)}{a^i}$$

where i is the octave (N total)
a^i scales the ith octave, controlling the amount of higher frequencies added
b^i controls the spacing (in frequency) between octaves

http://mrl.nyu.edu/~perlin/doc/oscar.html#noise

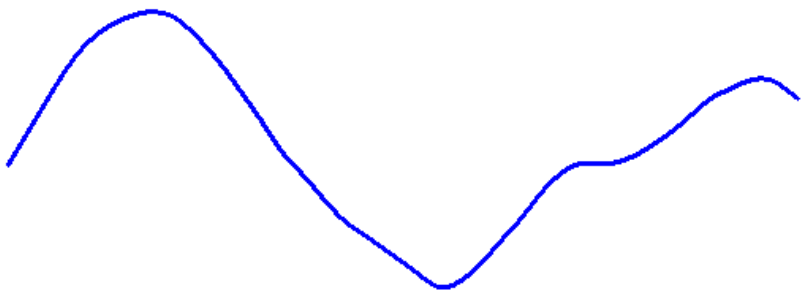
Perlin noise

- The result is a superposition of signals at different octaves



Control

Perlin(x, 2, 2, 4)



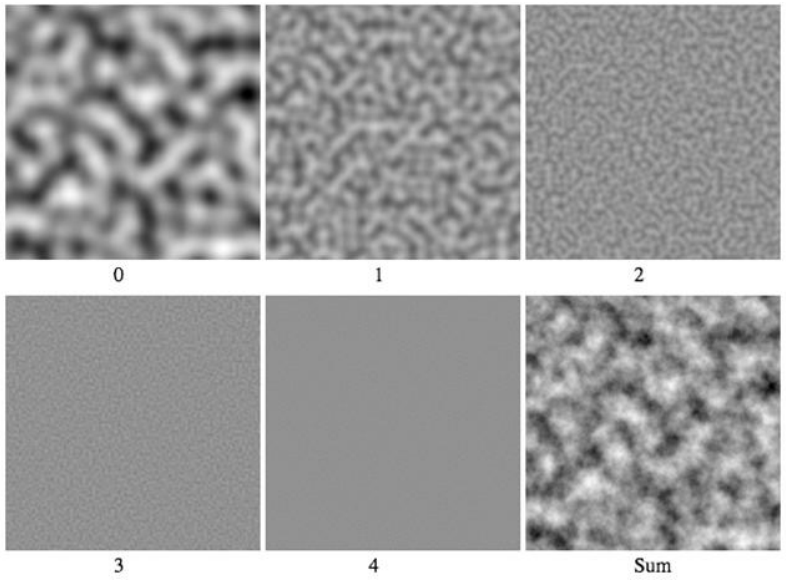
Perlin(x, 5, 2, 4)

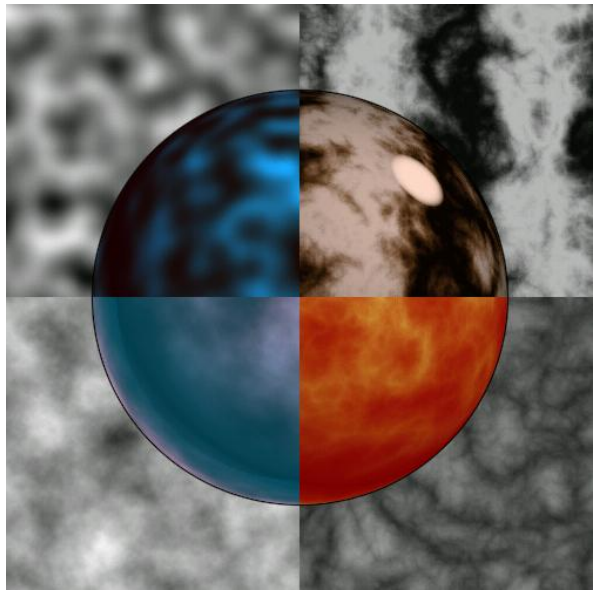
Increasing a: less signal from higher octaves

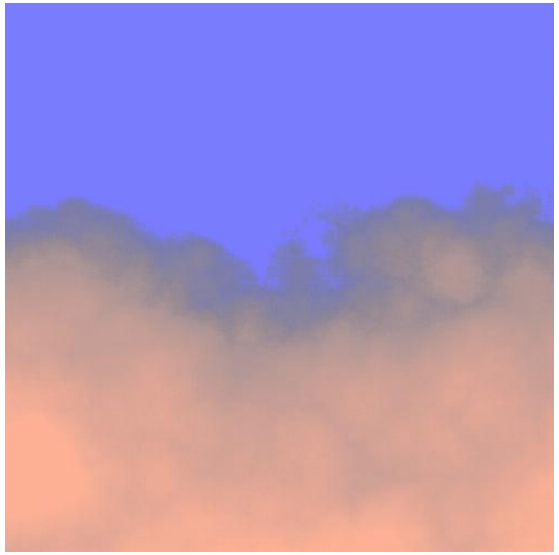
Perlin(x, 2, 5, 4)

Increasing b: higher frequencies for given octaves

Higher dimensions



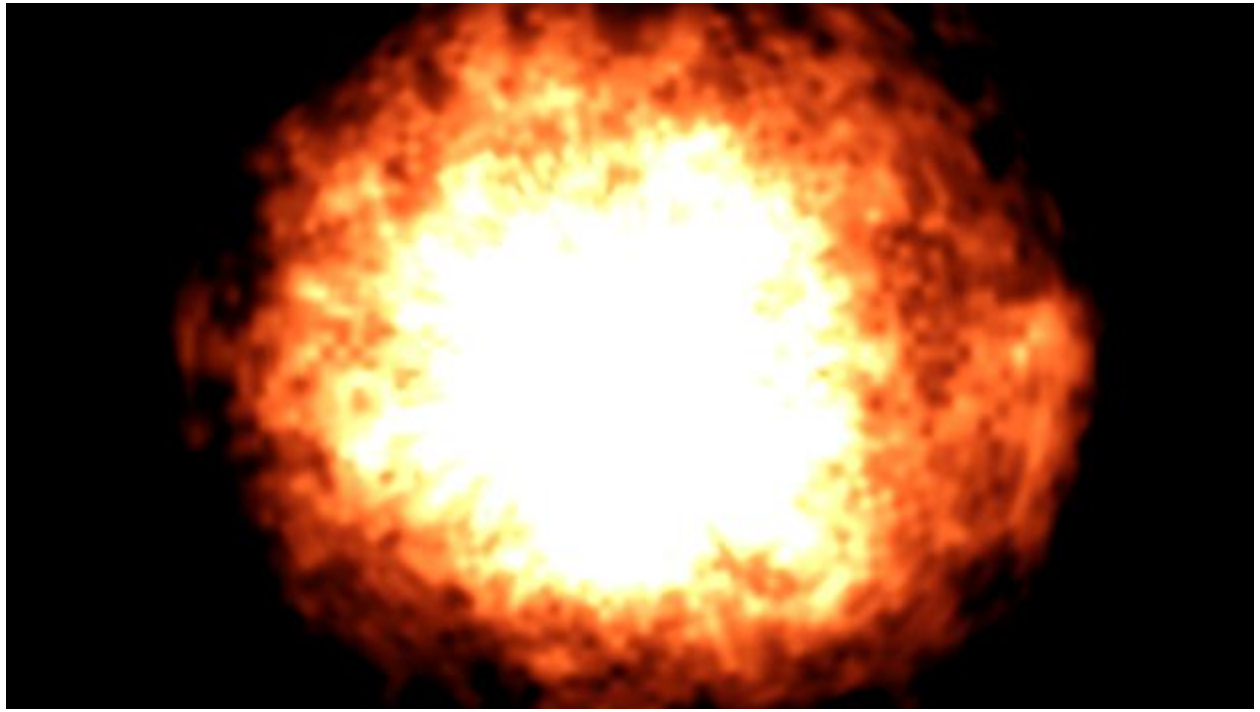




Noise code:

C
https://paulbourke.net/fractals/noise/
GLSL
https://github.com/ashima/webgl-noise/wiki

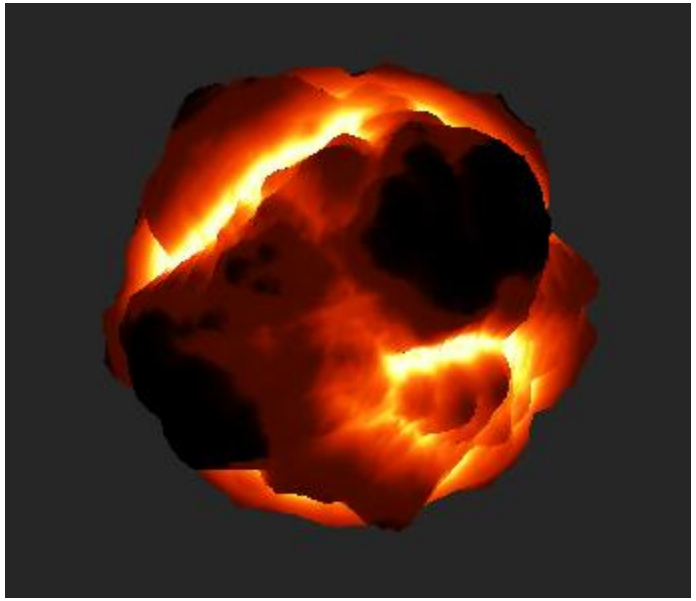
Demo (shadertoy)



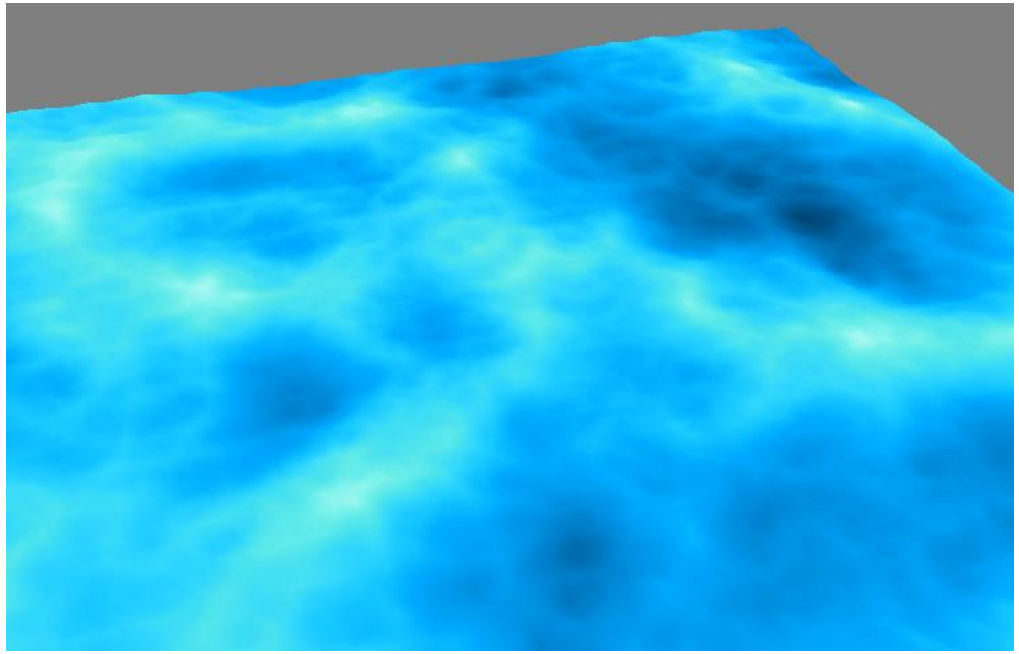
https://www.shadertoy.com/view/lsf3RH

Examples

- Fireball (based on sphere)
  - In the vertex shader:
    - Displace each vertex a distance d along its normal using Perlin noise, which is a function of position and time.
    - Change the sphere radius as a function of time using 1D Perlin noise
  - In the fragment shader:
    - Look up colour in texture, based on d, so that less displaced points are yellowish and more displaced points are dark



http://www.clicktorelease.com/code/perlin/explosion.html

Examples

- Water (based on heightmap terrain)
  - In the vertex shader:
    - Displace each vertex a distance d along its normal using Perlin noise, which is a function of position and time.
  - In the fragment shader:
    - Look up colour in texture, based on d, so that less displaced points are dark blue and more displaced points are white-ish